

Jenkins CI/CD Practical Handbook

Running Maven Builds, Test Reports, Code Coverage, Build Triggers, Pipeline Libraries, Jenkins Agents, and Deployment Flows

Audience	Objective
B.Tech / M.Tech / DevOps students	Practice Jenkins CI/CD from basic Maven build to full deployment pipeline.
Tools	Java, Maven, GitHub, Jenkins, Docker, Linux server/cloud VM

Practice rule: Complete each lab one by one. Do not jump directly to final CI/CD pipeline until Maven build, test reports, and deployment are working separately.

- 1. Lab Environment and Prerequisites**
- 2. Create a Maven Sample Project**
- 3. Push Project to GitHub**
- 4. Configure Jenkins Tools**
- 5. Practical 1 - Maven Build Pipeline**
- 6. Practical 2 - Unit Test Reports**
- 7. Practical 3 - Code Coverage with JaCoCo**
- 8. Practical 4 - Trigger Build using pollSCM**
- 9. Practical 5 - Trigger Build using GitHub Webhook**
- 10. Practical 6 - Pipeline Shared Library**
- 11. Practical 7 - Jenkins SSH/SFTP Agent Deployment**
- 12. Practical 8 - Container-Based Jenkins Agent**
- 13. Practical 9 - Deploy to Cloud/Server**
- 14. Practical 10 - Complete CI/CD Pipeline**
- 15. Troubleshooting and Viva Questions**

1. Lab Environment and Prerequisites

- Operating system: Windows, macOS, or Linux. Jenkins server may run locally or on a VM.
- Java JDK 17 or later installed.
- Apache Maven installed.
- Git and GitHub account available.
- Docker Desktop installed for container-agent practical.
- One Linux server or cloud VM for deployment practical.

Check versions

```
java -version
mvn -version
git --version
docker --version
```

Install suggested Jenkins plugins

Plugin	Purpose
Pipeline	Write Jenkinsfile based pipelines
Git	Clone project from GitHub
Maven Integration	Maven support in Jenkins
JUnit	Publish test results
JaCoCo / HTML Publisher	Show coverage report
SSH Agent	Deploy using private key
Docker Pipeline	Run builds inside Docker container
GitHub Integration	Webhook build trigger

2. Create a Maven Sample Project

Step 1: Open terminal and create a workspace folder.

Step 2: Generate a simple Maven Java project.

Step 3: Run Maven test and package commands locally.

Step 4: Confirm JAR file is created inside target folder.

```
mkdir devops-lab
cd devops-lab
mvn archetype:generate \
-DgroupId=com.demo \
-DartifactId=jenkins-maven-demo \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false

cd jenkins-maven-demo
mvn clean test
mvn clean package
ls target
```

Expected output

- BUILD SUCCESS should appear.
- target folder should contain .jar file.
- target/surefire-reports should contain XML test report files.

3. Push Project to GitHub

Step 1: Create a new GitHub repository named jenkins-maven-demo.

Step 2: Inside project folder, initialize Git.

Step 3: Add files, commit, connect remote repository, and push code.

```
git init
git add .
git commit -m "Initial Maven project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/jenkins-maven-demo.git
git push -u origin main
```

If remote origin already exists, correct it using:

```
git remote -v
git remote remove origin
git remote add origin https://github.com/YOUR-USERNAME/jenkins-maven-demo.git
git push -u origin main
```

4. Configure Jenkins Tools

Step 1: Open Jenkins dashboard.

Step 2: Go to Manage Jenkins -> Tools.

Step 3: Add JDK installation name as JDK-17.

Step 4: Add Maven installation name as Maven-3.

Step 5: Save configuration.

Tool	Example Name in Jenkinsfile
JDK	JDK-17
Maven	Maven-3

5. Practical 1 - Maven Build Pipeline

Step 1: Create a file named Jenkinsfile in the root of Maven project.

Step 2: Paste the pipeline code below.

Step 3: Commit and push Jenkinsfile to GitHub.

Step 4: Create Jenkins Pipeline job using Pipeline script from SCM.

Step 5: Click Build Now and check console output.

```
pipeline {
    agent any

    tools {
        jdk 'JDK-17'
        maven 'Maven-3'
    }

    stages {
        stage('Clone Code') {
```

```

        steps {
            git branch: 'main', url: 'https://github.com/YOUR-USERNAME/jenkins-maven-demo.git'
        }
    }

    stage('Maven Build') {
        steps {
            sh 'mvn clean package'
        }
    }
}

```

- Expected result: Jenkins console shows BUILD SUCCESS.
- Artifact should be available in target folder during workspace browsing.

6. Practical 2 - Unit Test Reports

Step 1: Modify Jenkinsfile to run Maven tests.

Step 2: Use post block to publish JUnit reports.

Step 3: Run the build.

Step 4: Open build page and click Test Result.

```

pipeline {
    agent any

    tools {
        jdk 'JDK-17'
        maven 'Maven-3'
    }

    stages {
        stage('Test') {
            steps {
                sh 'mvn clean test'
            }
        }
    }

    post {
        always {
            junit 'target/surefire-reports/*.xml'
        }
    }
}

```

Folder/File	Meaning
target/surefire-reports/*.xml	JUnit XML test result files
Test Result link	Jenkins test report dashboard
Failed tests	Visible with error trace in Jenkins

7. Practical 3 - Code Coverage with JaCoCo

Step 1: Open pom.xml.

Step 2: Add JaCoCo Maven plugin inside build/plugins.

Step 3: Run mvn clean test locally.

Step 4: Confirm target/site/jacoco/index.html is generated.

Step 5: Publish report in Jenkins using HTML Publisher.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.12</version>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
pipeline {
  agent any

  stages {
    stage('Test with Coverage') {
      steps {
        sh 'mvn clean test'
      }
    }
  }

  post {
    always {
      junit 'target/surefire-reports/*.xml'
      publishHTML([
        reportDir: 'target/site/jacoco',
        reportFiles: 'index.html',
        reportName: 'JaCoCo Code Coverage Report',
        keepAll: true,
        alwaysLinkToLastBuild: true,
        allowMissing: false
      ])
    }
  }
}
```

8. Practical 4 - Trigger Build using pollSCM

Step 1: Open Jenkinsfile.

Step 2: Add triggers block.

Step 3: Push change to GitHub.

Step 4: Wait for Jenkins to check repository.

Step 5: Build starts only if new commit is detected.

```
pipeline {
    agent any

    triggers {
        pollSCM('H/2 * * * *')
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

Expression	Meaning
H/2 *****	Check SCM approximately every 2 minutes
H/5 *****	Check approximately every 5 minutes
pollSCM	Jenkins pulls repository metadata and detects changes

9. Practical 5 - Trigger Build using GitHub Webhook

Step 1: In Jenkins job, enable GitHub hook trigger for GITScm polling or use githubPush trigger.

Step 2: In GitHub repository, open Settings -> Webhooks -> Add webhook.

Step 3: Enter Payload URL as <http://YOUR-JENKINS-URL/github-webhook/>.

Step 4: Select application/json.

Step 5: Select Just the push event.

Step 6: Save webhook and push a new commit.

```
pipeline {
    agent any

    triggers {
        githubPush()
    }

    stages {
        stage('Build from GitHub Webhook') {
            steps {
                sh 'mvn clean package'
            }
        }
    }
}
```

- Webhook is faster than pollSCM because GitHub notifies Jenkins immediately.
- For local Jenkins, use ngrok or deploy Jenkins on public IP so GitHub can reach it.

10. Practical 6 - Pipeline Shared Library

Step 1: Create a new GitHub repository named jenkins-shared-library.

Step 2: Create folder vars.

Step 3: Inside vars, create mavenBuild.groovy.

Step 4: Configure library in Jenkins: Manage Jenkins -> System -> Global Pipeline Libraries.

Step 5: Use @Library annotation in Jenkinsfile.

```
jenkins-shared-library/
├─ vars/
│   ├── mavenBuild.groovy
│   ├── publishReports.groovy
│   └─ deployJar.groovy
// vars/mavenBuild.groovy
def call() {
    sh 'mvn clean package'
}
// vars/publishReports.groovy
def call() {
    junit 'target/surefire-reports/*.xml'
    publishHTML([
        reportDir: 'target/site/jacoco',
        reportFiles: 'index.html',
        reportName: 'JaCoCo Report',
        keepAll: true,
        alwaysLinkToLastBuild: true,
        allowMissing: true
    ])
}
@Library('my-shared-lib') _

pipeline {
    agent any

    stages {
        stage('Build using Library') {
            steps {
                mavenBuild()
            }
        }
    }

    post {
        always {
            publishReports()
        }
    }
}
```

11. Practical 7 - Jenkins SSH/SFTP Agent Deployment

Step 1: Create Linux server or cloud VM.

Step 2: Install Java on server.

Step 3: Generate SSH key or use existing private key.

Step 4: Add private key in Jenkins credentials with ID server-ssh-key.

Step 5: Copy JAR using scp and run it using ssh command.

```
# On deployment server
sudo apt update
sudo apt install openjdk-17-jdk -y
java -version
mkdir -p /home/ubuntu/apps
pipeline {
    agent any

    stages {
        stage('Package') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Deploy using SSH') {
            steps {
                sshagent(['server-ssh-key']) {
                    sh '''
                        scp -o StrictHostKeyChecking=no target/*.jar
                        ubuntu@SERVER_IP:/home/ubuntu/apps/app.jar
                        ssh -o StrictHostKeyChecking=no ubuntu@SERVER_IP "pkill -f app.jar || true"
                        ssh -o StrictHostKeyChecking=no ubuntu@SERVER_IP "nohup java -jar
                        /home/ubuntu/apps/app.jar > /home/ubuntu/apps/app.log 2>&1 &"
                        '''
                }
            }
        }
    }
}
```

- SCP copies the file to server.
- SSH runs commands remotely.
- nohup keeps application running after SSH logout.
- pkill stops the old version before new deployment.

12. Practical 8 - Container-Based Jenkins Agent

Step 1: Install Docker on Jenkins server.

Step 2: Install Docker Pipeline plugin.

Step 3: Ensure Jenkins user can run docker command.

Step 4: Use Maven Docker image as build agent.

Step 5: Run pipeline and verify Maven works without local Maven installation.

```
pipeline {
    agent {
        docker {
            image 'maven:3.9.6-eclipse-temurin-17'
            args '-v $HOME/.m2:/root/.m2'
        }
    }

    stages {
```

```

    stage('Build in Maven Container') {
        steps {
            sh 'mvn clean package'
        }
    }

    stage('Test in Container') {
        steps {
            sh 'mvn test'
        }
    }
}
}

```

- Container agent gives clean and repeatable environment.
- Maven image already contains Maven and JDK.
- .m2 volume reduces dependency download time.

13. Practical 9 - Deploy to Server or Cloud

Step 1: Create cloud VM on AWS, Azure, GCP, or local Linux machine.

Step 2: Open application port in firewall/security group.

Step 3: Install Java or Docker depending on deployment type.

Step 4: Copy artifact from Jenkins.

Step 5: Start application and verify using browser or curl.

```

# Server-side check
ps aux | grep app.jar
curl http://SERVER_IP:8080
cat /home/ubuntu/apps/app.log

```

Docker-based deployment option

```

# Dockerfile
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
pipeline {
    agent any

    stages {
        stage('Build JAR') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t jenkins-maven-demo:latest .'
            }
        }

        stage('Run Container') {
            steps {
                sh 'docker rm -f demo-app || true'
            }
        }
    }
}

```

```

        sh 'docker run -d --name demo-app -p 8080:8080 jenkins-maven-demo:latest'
    }
}
}
}

```

14. Practical 10 - Complete CI/CD Pipeline

Step 1: Clone code from GitHub.

Step 2: Compile Maven project.

Step 3: Run unit tests.

Step 4: Generate test report.

Step 5: Generate code coverage report.

Step 6: Package JAR file.

Step 7: Deploy JAR to server.

Step 8: Show success/failure message.

```

pipeline {
    agent any

    tools {
        jdk 'JDK-17'
        maven 'Maven-3'
    }

    triggers {
        githubPush()
        // pollSCM('H/5 * * * *')
    }

    environment {
        SERVER_USER = 'ubuntu'
        SERVER_IP = 'YOUR_SERVER_IP'
        APP_PATH = '/home/ubuntu/apps/app.jar'
    }

    stages {
        stage('Clone') {
            steps {
                git branch: 'main', url: 'https://github.com/YOUR-USERNAME/jenkins-maven-demo.git'
            }
        }

        stage('Compile') {
            steps {
                sh 'mvn clean compile'
            }
        }

        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }
    }
}

```

```

stage('Package') {
    steps {
        sh 'mvn package -DskipTests'
    }
}

stage('Deploy') {
    steps {
        sshagent(['server-ssh-key']) {
            sh '''
                scp -o StrictHostKeyChecking=no target/*.jar ${SERVER_USER}@${SERVER_IP}:${APP_PATH}
                ssh -o StrictHostKeyChecking=no ${SERVER_USER}@${SERVER_IP} "pkill -f app.jar || true"
                ssh -o StrictHostKeyChecking=no ${SERVER_USER}@${SERVER_IP} "nohup java -jar $
{APP_PATH} > /home/ubuntu/apps/app.log 2>&1 &"
            '''
        }
    }
}

post {
    always {
        junit 'target/surefire-reports/*.xml'
        publishHTML([
            reportDir: 'target/site/jacoco',
            reportFiles: 'index.html',
            reportName: 'JaCoCo Coverage Report',
            keepAll: true,
            alwaysLinkToLastBuild: true,
            allowMissing: true
        ])
    }

    success {
        echo 'CI/CD pipeline completed successfully.'
    }

    failure {
        echo 'CI/CD pipeline failed. Check console output.'
    }
}
}

```

15. Troubleshooting and Viva Questions

Problem	Reason	Solution
mvn command not found	Maven not installed or tool not configured	Configure Maven in Manage Jenkins -> Tools
No test report found	Wrong report path or tests not run	Use target/surefire-reports/*.xml
Webhook not triggering	Jenkins not reachable from GitHub	Use public URL or ngrok
SSH permission denied	Wrong key or username	Check credential ID, username, and server authorized keys
Docker permission denied	Jenkins user cannot access Docker	Add Jenkins user to docker group
Port not opening	Firewall/security group blocked	Open 8080 or required app port

Common Viva Questions

- What is Jenkins pipeline?
- What is the difference between pollSCM and webhook?

- Why do we use Maven in Jenkins?
- What is JUnit report?
- What is JaCoCo code coverage?
- Why are shared libraries useful?
- What is a Jenkins agent?
- Difference between SSH agent and Docker agent?
- What is CI/CD?
- Why do we use post block in Jenkinsfile?

Student final checklist

- Maven project builds locally.
- Project pushed to GitHub.
- Jenkins pipeline job created.
- Maven build successful.
- JUnit test report visible.
- JaCoCo coverage report visible.
- pollSCM trigger tested.
- GitHub webhook tested.
- Shared library tested.
- SSH deployment tested.
- Docker/container agent tested.
- Complete CI/CD pipeline executed.